J.P. MORGAN PAYMENT INTEGRATION FOR SALESFORCE PWA KIT

Implementation Guide

DOCUMENT CONTROL

Document Title	J.P. Morgan Payment Integration for Salesforce PWA Kit — Implementation Guide
Document Version	1.0
Document Status	Draft
Last Updated (YYYY-MM-DD)	28-April-2026
Package Version	1.0.0
Compatibility	PWA Kit 3.x, React 17+/18+
Supported Cards	Visa, Mastercard, American Express, Discover
Supported Currency	USD, CAD
Supported Payment Methods	Credit Card, Google Pay, Apple Pay
Supported Locale	en_US, en_CA (Supports Multi-locale)

REVISION HISTORY

VERSION	DATE	AUTHOR	SUMMARY OF CHANGES
1.0	28-April 2026	JPMC	Initial release

TABLE OF CONTENTS

1. PURPOSE AND SCOPE

1.1 Purpose

This guide provides step-by-step instructions to integrate the J.P. Morgan Payment library (`@jpmorgan/jpmorgan-salesforce-pwa`) into a Salesforce Commerce Cloud PWA Kit storefront.

1.2 Scope

In scope

- Credit card payments with PIE encryption (verify, authorize, capture, refund, void)

- Google Pay integration (checkout and cart express)

- Apple Pay integration (checkout and cart express)

- Fraud check with Kount device fingerprinting

- Multi-merchant/ Multi-locale configuration

- Server-side order confirmation and status updates

2. OVERVIEW

The `@jpmorgan/jpmorgan-salesforce-pwa` library provides a plug-and-play payment integration for PWA Kit storefronts. It handles:

2.1 Key Capabilities

Card Payments - Authorize, capture, refund, void for Visa, Mastercard, Amex, Discover

PIE Encryption - Encrypts card data in browser before server transmission (PCI scope reduction)

SafeTech Tokenization- Replaces PAN with tokens for secure storage

Google Pay - Token-based payments on checkout page and cart express checkout

Apple Pay - Safari-based payments on checkout page and cart express checkout

Fraud Check - Kount device fingerprinting for risk assessment

Multi-MID - Different merchant accounts per locale

2.2 Process Flow Credit Card (High Level)

1. Shopper enters card details at checkout.

2. PIE scripts encrypt card data in the shopper's browser.

3. PWA app submits encrypted payment payload for authorization.

4. J.P. Morgan returns authorization outcomes and SafeTech token.

5. Capture occurs immediately or later depending on configured capture method.

*****Why does this matter? ** - Because the real card number is encrypted before it even leaves the customer's browser. This reduces the security burden and helps with PCI compliance.***

2.3 Process Flow Google Pay (High Level)

1. Shopper taps Google Pay button

2. Sends payment request to Google Pay

3. Google shows payment details. Customer uses their saved card in Google account

4. Payment requests are sent using encrypted payment data received from Google to J.P. Morgan

5. Capture occurs immediately or later depending on configured capture method.

2.4 Process Flow Apple Pay (High Level)

1. Shopper taps Apple Pay button

2. Sends payment request to Apple Pay

3. Apple shows payment details. Shopper approves with Face ID or Touch ID.

4. Payment requests are sent using encrypted payment data received from Apple to J.P. Morgan

5. Capture occurs immediately or later depending on configured capture method.

3. SOLUTION COMPONENTS

Think of this integration like building blocks that work together. Here is what each piece does:

****What is a PWA Bundle? *****

In Salesforce Commerce Cloud, a "PWA Bundle" is a finalized, compressed package of storefront code generated by the PWA Kit that is deployed to Managed Runtime (MRT) to deliver a modern, headless, and mobile-first shopping experience.

****What is a cartridge? *****

In Salesforce Commerce Cloud, a "cartridge" is like an app or plugin. You install it on your site to add new features.

Cartridge	Purpose
`int_jpmc_core`	Core payment logic (services, authentication, helpers)
`bm_jpmc`	Business Manager extensions (CSC payment actions, tools)

SFCC Business Manager — The Control Panel

****Business Manager***** is the admin area of your Salesforce Commerce Cloud site. Think of it as the "back office" where you manage products, orders, and settings

4. PREREQUISITES AND REQUIREMENTS

4.1 Prerequisites

Follow below Salesforce documentations and complete the PWA setup.

https://developer.salesforce.com/docs/commerce/pwa-kit-managed-runtime/guide/pwa-get-set-up.html

https://developer.salesforce.com/docs/commerce/pwa-kit-managed-runtime/guide/quick-start.html

Access to a running sandbox with RefArch Site available.

- *Reference: "Import the Storefront Reference Architecture (SFRA) Demo Site" section in https://help.salesforce.com/s/articleView?id=cc.b2c_site_import_export.htm&type=5*

Access to Salesforce Account Manager with roles https://account.demandware.com/dw/oidc/authorization/openAm

- *Commerce Cloud Developer Experience > Sandbox API User*
- *Commerce Cloud Managed Runtime > Managed Runtime User*
- *eComPlatform > Business Manager Administrator*
- *Shopper Admin API > SLAS Organization Administrator*

Access to Managed Run Time: https://runtime.commercecloud.com/

Apple Pay Developer Portal access: https://developer.apple.com/

4.2 Environment Requirements

PWA Kit Version - 3.x or higher

React Version - 17.x or 18.x

Node.js - 18.x or higher

VS Code with Prophet (Debugger and Uploader for SFCC) Plugin

4.3 Inputs Required from J.P. Morgan Payments Representative

- Merchant ID
- Client ID
- Token URI (OAuth2 token endpoint)
- Resource ID (OAuth2 resource identifier)
- PIE Merchant ID for Cards
- Google Pay Gateway Merchant ID for Google Pay
- Google Pay Gateway name for Google Pay
- Payment Processing Certificate for Apple Pay
- SafeTech Client ID for Fraud Detection Feature
- Signed Certificate – (If you have followed one of the paths listed below)

4.4 Certificate Signing Request Generation Steps

** Either Merchant can generate a CSR via OpenSSL and share with J.P Morgan to receive a signed certificate or share a signed certificate obtained from one of the certificate authorities listed in the J.P Morgan documentation.

[oauth-authentication]

Check with J.P Morgan for any clarifications on the certificate.

1. CSR creation steps:

   Use below OpenSSL commands to Generate Private Key and CSR after filling necessary details. (Assuming OpenSSL is available)

   Generate Private Key:
   openssl req -new -newkey rsa:2048 -nodes -out jpmc_file.txt -keyout jpmc_private_key.key -subj "/C=US/ST=XXXXXX/L=XXXXXX/O=XXXXX/OU=XXXXX/CN=XXXXX"

Generate CSR:
openssl req -new -key jpmc_private_key.key -out jpmc_csr_file.csr -subj "/C=US/ST=XXXXXX/L=XXXXXX/O=XXXXX/OU=XXXXX/CN=XXXXX"

Country Name (C): Use a 2-letter ISO code (e.g., "US").
State or Province Name (ST): Full name, do not abbreviate (e.g., "California").
Locality Name (L): City or town (e.g., "San Francisco").
Organization Name (O): The legal name of your company.
Organizational Unit (OU): Department name (e.g., "IT") or leave blank.
Common Name (CN): The fully qualified domain name (FQDN) you want to secure (e.g., www.example.com).

-new: Indicates a new CSR request.
-newkey rsa:2048: Generates a new 2048-bit RSA key.
-nodes: Short for "no DES," this prevents the private key from being encrypted with a password (common for web servers to allow automated restarts).
-keyout: The filename for your private key.
-out: The filename for your CSR.

2. Keep Private Key safe with you. Share only CSR with JPMC to receive a Signed Certificate.
3. Once signed certificate is received, Convert Cert and Private Key into base64 format. [Refer Appendix D for Base 64 encoding Steps]
   These are required to update in the environment variables in the later steps.

4.5 Steps to Update Required Scopes in SLAS Client

Note: Assuming SLAS Client is created while following prerequisites. Ensure prerequisites are completed before starting this section. Otherwise, you should complete and come back.

1. Log in to the SLAS Admin UI at this URL, https://short-code.api.commercecloud.salesforce.com/shopper/auth-admin/v1/sso/login

   Replace short-code with the short code for your B2C Commerce instances.

For example: https://sandbox-001.api.commercecloud.salesforce.com/shopper/auth-admin/v1/sso/login

You will find short code in the BM
Administration | Site Development | Salesforce Commerce API Settings

2. Select Clients.
3. Select the SLAS Client created for PWA in Prerequisites.
4. Click Edit, add below scopes in the Scopes section (separated by space) and Save

   sfcc.shopper-baskets.rw

   sfcc.shopper-orders.rw

   sfcc.shopper-products

   sfcc.shopper-search

   sfcc.shopper-custom-objects

5. SLAS Client ID is required to update in the environment variables in the later steps. Refer Appendix C for the steps to create SLAS Client.

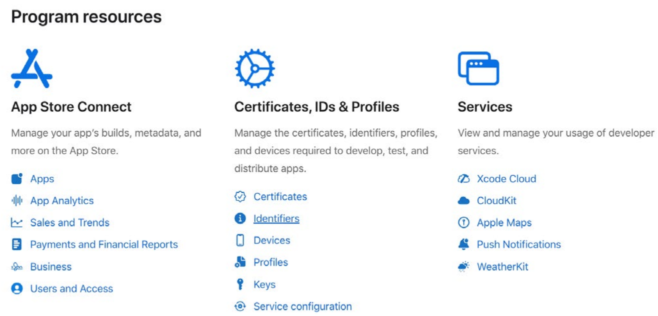
4.6 Steps to Create API Client with Required Scopes

- Follow the documentation and create API Client https://help.salesforce.com/s/articleView?id=cc.b2c_account_manager_add_api_client_id.htm&type=5
- Ensure below configurations are selected:
  - Choose Authentication Type as **Confidential**
  - Choose your **organization**
  - Click Roles and select **Salesforce Commerce API**
  - Add allowed scopes
    sfcc.preferences
    sfcc.orders.rw
  - Choose Token Endpoint Auth Method as **client_secret_post**
- API Client and Password (Secret) are required to update in the environment variables in the later steps.

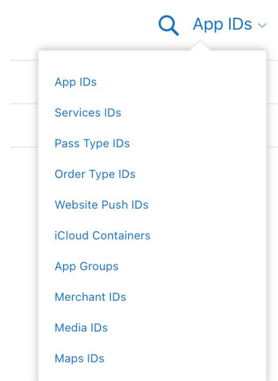
5. APPLE PAY DEVELOPER PORTAL

5.1 Create Apple Pay Merchant ID

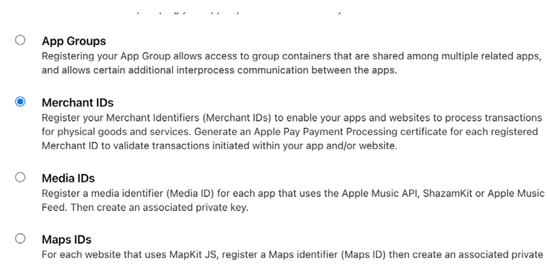
- Login to https://developer.apple.com/account
- Click on Identifiers under Certificates, IDs & Profiles



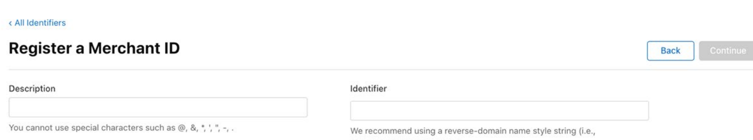
- Choose Merchant IDs from the top right selector.



- Click on "+" icon next to Identifiers.
- Ensure Merchant IDs is selected and click Continue



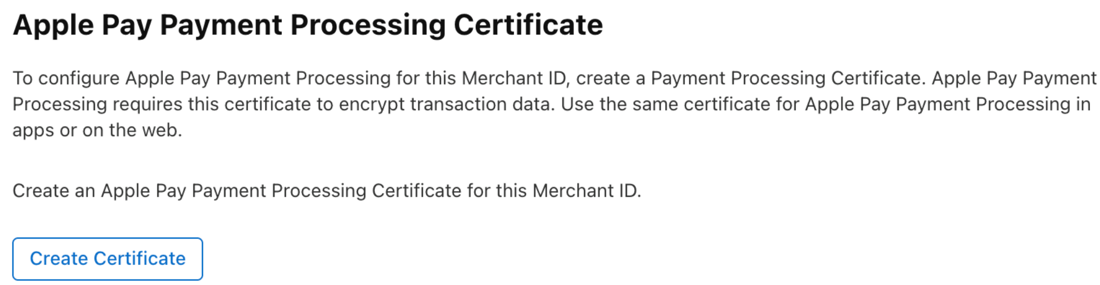
- Add Description and provide unique Apple pay identifier



- Click Continue and Register
- Share this Apple pay Merchant ID with JPMC.

5.2 Generate Payment Processing Certificate

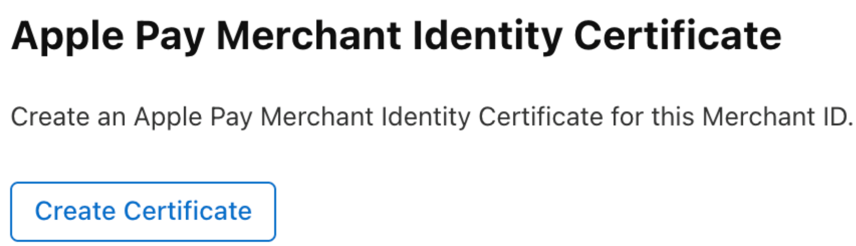
- Get CSR from JPMC, upload in the below section to obtain signed certificate from Apple – This certificate allows JPMC to decrypt the payment.



- Download and share the signed cert with Apple.

5.3 Generate Merchant Identity Certificate

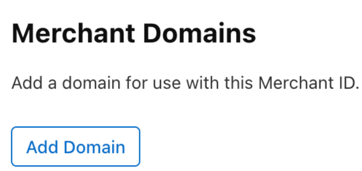
- Generate a CSR, upload in the below section to obtain signed certificate from Apple – This certificate allows Apple to validate Merchant.



- Convert Signed Cert into PEM format using below command
  openssl x509 -inform DER -in merchant_id.cer -out merchant_identity.pem
- Convert Private Key and PEM Cert into base64 format. [Refer Appendix D for Base 64 encoding Steps]

5.4 Merchant Domain verification step

- Click on Add Domain and provide Site Domain obtains from MRT and click Save.



Login into your MRT environment - https://runtime.commercecloud.com/login
Select Organization | Choose project | Choose environment
From left sidebar click on Environment Settings

Copy domain name from "Proxy Configs" section.

- Click on Download – This gives you a file *apple-developer-merchantid-domain-association.txt*.
- Place this in your PWA code base in the below location. *retail-react-app/overrides/app/static/apple-developer-merchantid-domain-association.txt*
- For Domain validation - Skip to "**Merchant Domain Verification - Apple**" section.

6. METADATA IMPORT

6.1 Purpose

Metadata import creates required services, site preferences, custom objects, and object extensions in Business Manager.

** This section involves importing XML files into your Salesforce site. If you are not comfortable with file imports, please ask your web developer or IT support person for help.

Goal: Tell the SFCC BM, about all the new data fields, services, jobs, payment-methods, custom processor and custom objects the J.P. Morgan cartridge uses.

File	What It Does
system-objecttype-extensions.xml	Creates all the JPMC configuration fields in Business Manager (like "JPMC Client ID," "Capture Method," etc.) and adds payment data fields to orders.
custom-objecttype-definitions.xml	Creates the behind-the-scenes data storage objects — access tokens, merchant configurations.
services.xml	Sets up the connection endpoints (URLs) that your store uses to communicate with J.P. Morgan's servers.
payment-methods.xml	Defines the available payment options (e.g., Credit Card, Google Pay Payments) visible at checkout and maps them to the appropriate custom processor.

payment-processors.xml	Registers the custom payment processors and maps them to the specific cartridge scripts responsible for handling payment hooks and transaction states.

6.2 Prepare the Import Archive

To prepare the JPMC integration metadata for your specific environment, follow the steps below within the "metadata" directory of the codebase.

Step 1: Configure the Site Directory

- *Navigate to the following path: metadata/site_import/sites/.*
- *Locate the folder named RefArch (the default reference architecture ID).*
- *Rename the RefArch folder to match your Target Site ID exactly as it is configured in Business Manager (e.g., YourBrandID).*

Step 2: Package for Site Import

- *Return to the "metadata" directory.*
- *Compress the site_import folder into a .zip archive. **Important**: Ensure the archive structure is correct for SFCC Site Import. The root of the zip should contain the sites, meta, service components.*

  For a reference on the required structure, you can inspect the existing sample at: metadata/site_import.zip.

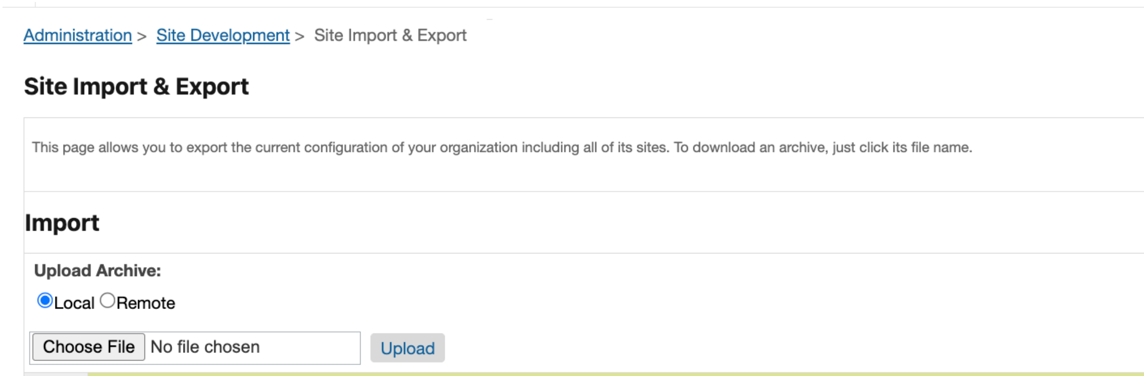
Step 3 - Upload and Import in Business Manager

1. **Administration** > **Site Development** > **Site Import & Export**

2. Under **Import**, click **Upload** and select `site_import.zip`.

3. Select the uploaded zip and click **Import**.

Step 4 - Confirm imports complete with **SUCCESS**.



Objects and Configurations Created

File	Creates
`services.xml`	HTTP services, credentials, service profile
`system-objecttype-extensions.xml`	Site preferences and groups, Order and Payment custom attributes
`custom-objecttype-definitions.xml`	merchant configurations
`payment-methods.xml`	JPMC Payment methods
`payment-processors.xml`	Custom processor JPMC_Payment

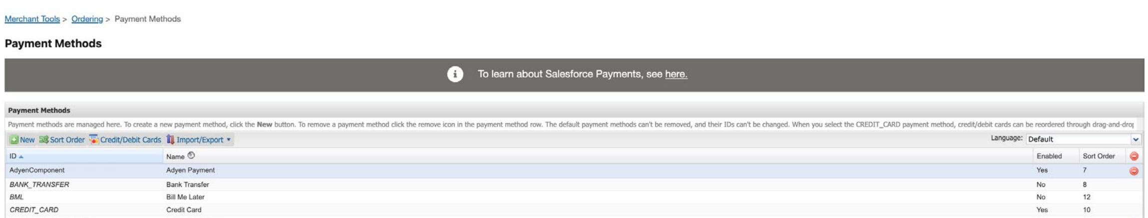
7. PAYMENT METHODS

Prerequisite: This section assumes you have successfully completed the Site Import process outlined in the Metadata Import section. The payment processors and methods must be imported.

Please verify the following configurations to ensure that checkout transactions are routed correctly to the J.P. Morgan gateway.

7.1 Ensure Credit Card is enabled in Payment Method

- *Merchant Tools > Ordering > Payment Methods*
- *Locate `CREDIT_CARD` (Has to be enabled)*
- *Ensure Enabled*
- *Verify/Set Payment Processor to `JPMC_Payment`*
- *Apply changes*



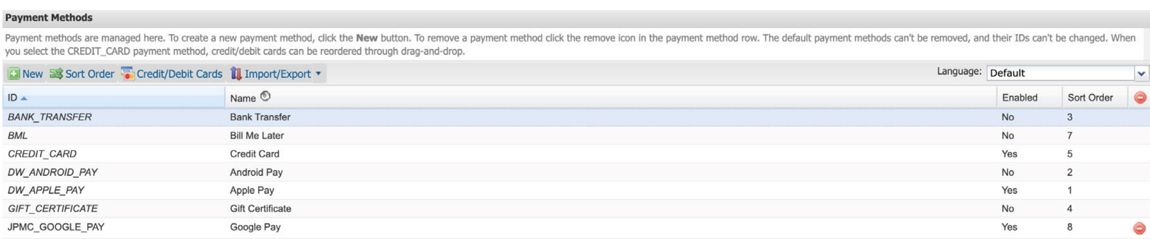
7.2 Verify Accepted Card Types

Card Type	Card Brand
Visa	Visa
Mastercard	Mastercard
Amex	American Express

Diners Club	Diners Club
Discover	Discover
JCB	JCB

7.3 Ensure Apple Pay and Google Pay are enabled in Payment Method

- *Merchant Tools > Ordering > Payment Methods*
- *Locate `DW_APPLE_PAY and JPMC_GOOGLE_PAY ` (Has to be enabled)*
- *Ensure Enabled*
- *Verify/ Set Payment Processor to `JPMC_Payment`*
- *Apply changes*



Payment Methods

Payment methods are managed here. To create a new payment method, click the **New** button. To remove a payment method click the remove icon in the payment method row. The default payment methods can't be removed, and their IDs can't be changed. When you select the CREDIT_CARD payment method, credit/debit cards can be reordered through drag-and-drop.

New Sort Order Credit/Debit Cards Import/Export Language: Default

ID	Name	Enabled	Sort Order
BANK_TRANSFER	Bank Transfer	No	3
BML	Bill Me Later	No	7
CREDIT_CARD	Credit Card	Yes	5
DW_ANDROID_PAY	Android Pay	No	2
DW_APPLE_PAY	Apple Pay	Yes	1
GIFT_CERTIFICATE	Gift Certificate	No	4
JPMC_GOOGLE_PAY	Google Pay	Yes	8

Verifying the payment method setup –

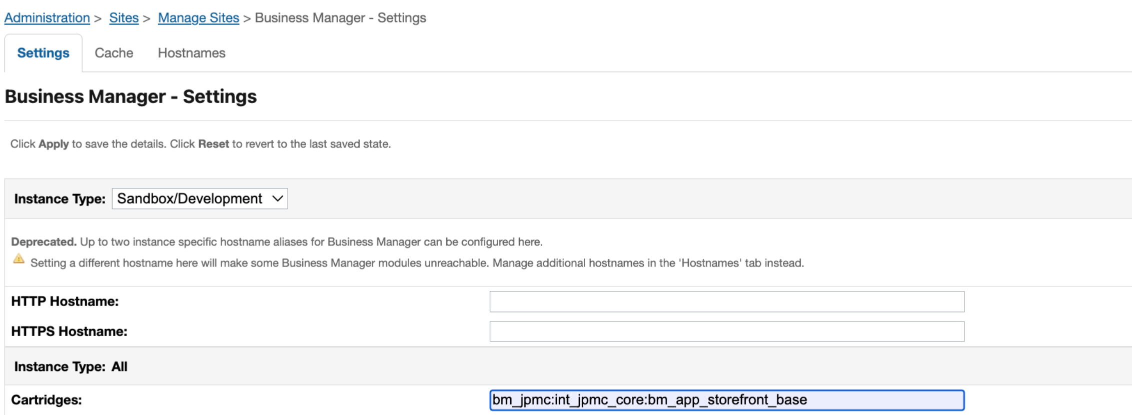
- Confirm `CREDIT_CARD` enabled with `JPMC_Payment` processor
- Confirm accepted card types align to target market
- If using Google Pay: Is `JPMC_GOOGLE_PAY` payment method enabled with `JPMC_Payment` processor
- If using Apple Pay: Is Apple Pay related site preferences are configured

8. CARTRIDGE SETUP (BM CONFIGURATION)

8.1 Update Business Manager Cartridge Path

- *Administration > Sites > Manage Sites*
- *Select Business Manager.*
- *Settings tab > Cartridges*
- *Add `bm_jpmc` and `int_jpmc_core` before `bm_app_storefront_base`*

  Like this: `bm_jpmc:int_jpmc_core:bm_app_storefront_base`
- *5. Click Apply.*



8.2 Verify Services are Available

1. Administration > Operations > Services

2. Confirm these services appear:

   1. Authentication & Security

      - Service: JPMCAccessToken

        - Purpose: Handles OAuth 2.0 authentication and bearer token generation.

    - Credential ID: JPMCAccessTokenCredentials

2. Order Lifecycle

    - Service: JPMCPaymentCapture
        - Purpose: Executes capture requests for authorized payments.
        - Credential ID: JPMCPaymentCaptureCredentials
    - Service: JPMCPaymentRefund
        - Purpose: Manages transaction refunds for processed orders.
        - Credential ID: JPMCPaymentRefundCredentials
    - Service: JPMCPaymentVoid
        - Purpose: Voids authorized payments prior to capture.
        - Credential ID: JPMCPaymentVoidCredentials

⚠ Important: Ensure CAT or Production URLs are configured in each credential. Current configurations have Mock URLs.

9. SITE PREFERENCES CONFIGURATION

9.1 Open Preference Groups

Configure the J.P. Morgan Commerce (JPMC) payment integration through Merchant Tools > Site Preferences > Custom Preferences.

This section outlines all configuration groups.

1. ***JPMC Payment Configuration*** *(15 preferences including fraud, tokenization, PIE URLs)*
2. ***JPMC PWA*** *(3 preferences)*
3. ***JPMC Apple Pay Configuration*** *(6 preferences)*
4. ***JPMC Google Pay Configuration*** *(9 preferences)*
5. ***JPMC Multi Merchant Settings*** *(1 master toggle)*
6. ***JPMC 3DS Settings*** *(1 master toggle)*

9.2 Configuration Groups

9.2.1 JPMC Payment Configuration

Navigate to:

Merchant Tools > Site Preferences > Custom Preferences > JPMC Payment Configuration

Core Credentials & Tokens

1. ***JPMC Client ID:*** *OAuth2 Client ID from JPMorgan. (String/Password)*
2. ***JPMC Merchant Id****: Merchant ID from JPMorgan (e.g., *YOUR_merchantId*). (String)*
3. ***JPMC Merchant Category Code****: Merchant Category Code from JPMorgan (4 Digited number)*
4. ***JPMC Tokenization Type****: Default tokenization type for transactions*
   (Enum: SAFETECH_TOKEN, NETWORK_TOKEN).

Default: SAFETECH_TOKEN.

5. ***JPMC Capture Method:***
   - *MANUAL: Authorization only; capture performed later via Commerce Cloud CSC.*
   - *DELAYED: Authorization with delayed auto-capture (120-minute window).*
   - *NOW: Immediate sale (authorize + capture in one request).*

Default: MANUAL.

Fraud & Verification

6. ***Enable JPMC Fraud Check (Handle):*** *Validates fraud during payment verification (basket stage). Declines block the transaction. (Boolean)*
   Default: NO
7. ***Enable JPMC Fraud Check (Authorize):*** *Validates fraud before payment authorization (order stage). (Boolean)*
   Default: NO
8. ***Safetech Fraud Client ID:*** *Safetech Fraud Web Client SDK Client ID (6-digit ID). Required if fraud check is enabled. (String)*
9. ***Safetech Fraud Environment:*** *Enum: TEST or PROD.*
   Default: TEST
10. ***Enable Address Verification Service (AVS):*** *Includes billing address in Verify Payment and Authorization payloads. (Boolean)*
    Default: NO

URLs & Encryption (PIE)

11. ***JPMC Resource ID: OAuth2 resource ID.*** *(Enum: Test/CAT, Production)*
    Default: TEST

12. ***JPMC PIE Encryption Library URL**: URL for JPMC Page Integrated Encryption (PIE) script.*
    Default: TEST

13. ***JPMC PIE Get Key Base URL**: Base URL for JPMC PIE getkey.js script.*
    Default: TEST

14. ***JPMC PIE Key: Page Encryption (PE) Key ID**: PE ID - Appended to construct the full URL. (String)*

Software & Platform Identifiers

15. ***JPMC Platform ID:** Optional platform identifier. (String)*

9.2.2 JPMC PWA

16. ***JPMC Environment:** JPMC environment used to determine the API host when JPMC API Host is not explicitly set. (Enum: sandbox, production)*
    Default: sandbox
17. ***JPMC API Host:** JPMC Payment API base URL (without https://). Overrides the host derived from JPMC Environment when set.*
    Examples: api-ms-test.payments.jpmorgan.com (sandbox), api-ms.payments.jpmorgan.com (production). (String)
18. ***JPMC Google Pay Allowed Shipping Countries:** Comma-separated list of ISO 3166-1 alpha-2 country codes for allowed Google Pay shipping destinations (e.g. US,CA,MX). Only addresses in listed countries will be selectable in the Google Pay sheet. If empty, defaults to the site country code. (String)*

9.2.3 JPMC Apple Pay Configuration

19. ***JPMC Apple Pay Enabled:*** *Master toggle to enable Apple Pay as a payment method. (Boolean)*
    Default: NO

20. ***JPMC Apple Pay Merchant ID:*** *Apple Pay merchant identifier registered with Apple (e.g. merchant.com.company.store). (String)*

21. ***JPMC Apple Pay Merchant Name:*** *Merchant name displayed to the customer in the Apple Pay payment sheet. (String)*

22. ***JPMC Apple Pay Country Code:*** *Two-letter ISO 3166 country code for the Apple Pay session (e.g. US). (String)*

23. ***JPMC Apple Pay Supported Networks:*** *Comma-separated list of card networks supported for Apple Pay (e.g. visa,masterCard,amex,discover). (String)*

24. ***JPMC Apple Pay Merchant Capabilities:*** *Comma-separated list of Apple Pay merchant capabilities (e.g. supports3DS,supportsCredit,supportsDebit). (String)*

Navigate to: **Merchant Tools > Site Preferences > Custom Preferences > JPMC Google Pay Configuration**

25. ***Google Pay Environment****: TEST or PRODUCTION.*
    Default: TEST

Choose `Test` while you're setting up and testing. Switch to `Production` only when you're ready for real customer payments.

26. ***Google Pay Gateway Merchant ID**: this is the merchant ID that J.P. Morgan (Chase) provided to you for Google Pay.*
27. ***Google Pay Merchant ID:** Your Merchant ID from the Google Pay Business Console. For `Test` mode: leave this blank. For `Production`: this is required.*
28. ***Google Pay Merchant Name**: The name your customers see in the Google Pay pop-up.*
29. ***Google Pay Gateway**: The payment gateway name, typically `chase`. Your J.P. Morgan representative will confirm.*
30. ***Google Pay Allowed Card Networks**: A comma-separated list of card types you accept. Enter: `VISA,MASTERCARD,AMEX,DISCOVER` (use exactly this format, all capitals, no spaces after commas)*
31. ***Google Pay Allowed Auth Methods:** A comma-separated list of how cards can be authenticated. Enter: `PAN_ONLY,CRYPTOGRAM_3DS` (use exactly this format).*
32. ***JPMC Google Pay Cart Enabled:** Enables Google Pay on Cart - <u>Default: NO</u>*
33. ***JPMC Google Pay PDP Enabled:** Enables Google Pay on PDP - <u>Default: NO</u>*

9.2.4 JPMC Multi Merchant Settings

Navigate to: **Merchant Tools > Site Preferences > Custom Preferences > JPMC Multi Merchant Settings**

34. ***Enable Multi-Merchant Configuration (Boolean):** DISABLED (False): Entire site uses single merchant config from Site Preferences.*

ENABLED (True): Uses per-locale/per-merchant configurations from custom objects (JPMCMerchantConfig) and cache. Site Preferences act as a fallback.

9.2.5 JPMC 3DS Settings

Navigate to: **Merchant Tools > Site Preferences > Custom Preferences > JPMC 3D Secure**

35. ***Enable JPMC 3D Secure (Boolean):***
    DISABLED (False): Disables 3DS Feature.
    ENABLED (True): Enables 3DS Feature.

10. MANAGED RUNTIME (MRT)

Note: Assuming you have MRT access with a project created. Ensure prerequisites are completed before starting this section.

Steps to create environment variables in the MRT.

- Login into your MRT environment - https://runtime.commercecloud.com/login
- Select Organization | Choose project | Choose environment
- From left sidebar click on Environment variables
- Click on “Add variable”
- Enter name, value and click on Add variable

Add all the below variables in the MRT instance.

MRT variables:

VARIABLE NAME	DESCRIPTION
JPMC_PRIVATE_KEY_BASE64	Base64-encoded JPMC private key
JPMC_CERTIFICATE_BASE64	Base64-encoded JPMC certificate
COMMERCE_API_CLIENT_ID_PRIVATE	API client ID
COMMERCE_API_CLIENT_SECRET	API client secret
COMMERCE_API_SHORT_CODE	SFCC short code
COMMERCE_API_ORG_ID	SFCC organization ID
COMMERCE_API_SITE_ID	SFCC site ID (e.g., `RefArch`)
SFCC_REALM_ID	SFCC realm ID
SFCC_INSTANCE_ID	SFCC instance ID
APPLE_PAY_MERCHANT_IDENTITY_CERT_BASE64	Base64-encoded Apple Pay merchant identity certificate
APPLE_PAY_MERCHANT_IDENTITY_KEY_BASE64	Base64-encoded Apple Pay merchant identity private key
JPMC_DEBUG	Enables verbose debug logging (Optional)

11. ARCHITECTURE

11.1 Package Structure

```
@jpmorgan/jpmorgan-salesforce-pwa
├── lib/
│   ├── index.js            # Main entry point
│   ├── client/             # Browser-safe exports
│   │   ├── context/        # JPMCCheckoutProvider
│   │   └── components/     # GooglePayButton, ApplePayButton
│   ├── hooks/              # React hooks
│   ├── services/           # API services (auth, PIE, SFCC)
│   ├── ssr/                # Server-side routes and middleware
│   └── utils/              # Constants, validators, formatters
└── dist/                   # Built output
```

11.2 Import Paths

````
```javascript

// Main entry (contains everything)
import { createJPMCHandler, JPMCCheckoutProvider } from
'@jpmorgan/jpmorgan-salesforce-pwa'

// SSR-only (Node.js server code)
import { registerJPMCEndpoints, jpmorganCSPMiddleware } from
'@jpmorgan/jpmorgan-salesforce-pwa/ssr'

// Client-only (React browser code)
import {
    JPMCCheckoutProvider,
    useJPMCCheckout,
    useJPMCPlaceOrder,
    GooglePayButton,
    ApplePayButton
} from '@jpmorgan/jpmorgan-salesforce-pwa/client'
```
````

12. INSTALLATION

****Technical Notice**:** The following steps involve SFCC PWA kit setup along with SFCC business manager file uploads (for creation of configurations), creating page overrides of default PWA kit pages and integrating it into the codebase. If you are not comfortable with this, please forward this section to your **web developer or IT support person**.

12.1 Install the Package

Install the Package in the retail-react-app project.

````
```bash
npm install @jpmorgan/jpmorgan-salesforce-pwa
```
````

12.2 Peer dependencies

The library requires these peer dependencies (already part of PWA Kit):

````
```json
{
  "peerDependencies": {
    "@salesforce/pwa-kit-runtime": ">=3.0.0",
    "react": ">=17.0.0"
  }
}
```
````

12.3 MRT deployment

- **Build your project** - Run `npm run build` in your retail-react-app directory to create the production bundle.
- **Push to MRT** - Run `npm run push` to deploy your bundle to Managed Runtime. This uploads your code to Salesforce's cloud infrastructure.

- **Verify deployment** - After deployment, test your storefront by placing a test order. Check MRT logs for any configuration errors using: `npx @salesforce/pwa-kit-dev@latest tail-logs --project <project-name> --environment <env-name>`

12.4 Merchant Domain Verification – Apple

- Login to https://developer.apple.com/account
- Click on Identifiers under Certificates, IDs & Profiles
- Choose Merchant IDs on the top right, click the Apple Merchant ID created.
- Scroll down to “Merchant Domain” section and click Verify
- Click on OK (You must see status verified).

12.5 Local testing (Optional)

Since the library is not published on **npm**, these steps can be used to simulate the behavior -
Step 1 – In the terminal/cmd, go to the library folder (root)

Step 2 – Run `npm run build; npm pack`, this will create a tarball file which can be installed like a **npm** published package.

Step 3 – Go to the project running on retail-react-app in terminal. Install the tarball file that was created with steps above. Use command to install - `npm install jpmorgan-jpmorgan-salesforce-pwa-1.0.0.tgz"` update the file location to where the tarball is located.

Step 4 – Build and start your project using - `npm run build; npm start`

Running app locally will also need a .env file with variables from MRT at root level of your project.

```
# File paths (for local development only - use BASE64 versions in MRT)
JPMC_PRIVATE_KEY_BASE64=<base64-encoded-private-key>
JPMC_CERTIFICATE_BASE64=<base64-encoded-cert>


# Private client credentials (different from public PWA Kit client)
COMMERCE_API_CLIENT_ID_PRIVATE=<private-api-client-id>
COMMERCE_API_CLIENT_SECRET=<api-client-secret>
COMMERCE_API_SHORT_CODE=<short-code>
COMMERCE_API_ORG_ID=<organization-id>
COMMERCE_API_SITE_ID=<Your-Site-ID>

# Required for Site Preferences API scope
SFCC_REALM_ID=<realm-id>
SFCC_INSTANCE_ID=<instance-id>
===========================================================================
# Apple Pay (REQUIRED if Apple Pay is enabled)
===========================================================================
APPLE_PAY_MERCHANT_IDENTITY_CERT_BASE64=<base64-encoded-cert>
APPLE_PAY_MERCHANT_IDENTITY_KEY_BASE64=<base64-encoded-key>
# APPLE_PAY_MERCHANT_IDENTITY_PASSPHRASE=<optional-passphrase>
```

13. INTEGRATION STEPS

13.1 Integration prerequisites

1. Package is installed with steps mentioned.
2. Metadata is imported in the business manager to create custom attributes and site preferences.
3. Environment variables are created and have correct values.

13.2 Integration steps and files

Order	File	Purpose
1	`ssr.js`	Server setup, API routes, CSP
2	`routes.jsx`	Route overrides
3	`checkout/index.jsx`	Provider wrapper, place order
4	`checkout/partials/payment.jsx`	Card form, wallet buttons
5	`checkout/confirmation.jsx`	Wallet payment display
6	`cart/index.jsx`	Cart express checkout
7	`cart/partials/cart-cta.jsx`	Cart express buttons

14. SSR INTEGRATION

14.1 Modify `ssr.js`

The SSR layer is where you register JPMC API routes and CSP headers. This is required for payment processing

Step 1:
Import these required files –

- **bodyParser** is a dependency and is used for POST request parsing.
- **registerJPMCEndpoints** registers all endpoints used for Payments
- **jpmorganCSPMiddleware** adds content security policies for PIE SDK and wallets

```
import bodyParser from 'body-parser'
import {
    registerJPMCEndpoints,
    jpmorganCSPMiddleware
} from '@jpmorgan/jpmorgan-salesforce-pwa/ssr'
```

Step 2:

Add these in your runtimeHandler –

- **app.use(bodyParser.json())** - Enables Express to parse JSON bodies from POST requests (required for payment API endpoints).

- **app.use(jpmorganCSPMiddleware())** - Adds Content-Security-Policy headers allowing PIE SDK scripts from *.chasepaymentech.com and Google Pay scripts to load.

- **commerceConfig** - Credentials for Commerce API, used for server-side order operations like updating order status after payment.

- **registerJPMCEndpoints(app, runtime, {...})** - Registers all JPMC API routes (/api/jpmorgan/encrypt, /authorize, /google-pay/*, /apple-pay/*, /config) on the Express server.
- **Apple Pay domain verification file** - Apple fetches this during merchant validation setup. Served at both the canonical extensionless path (Apple's validator) and .txt (browser access). Source: Apple Developer Portal ->

Merchant IDs -> merchant.com.jpmc.sfra -> Merchant Domains.

```
    app.use(bodyParser.json())
    app.use(jpmorganCSPMiddleware())

    const commerceConfig = {
        clientId: process.env. COMMERCE_API_CLIENT_ID_PRIVATE
        clientSecret: process.env.JPMC_COMMERCE_CLIENT_SECRET
    }

    registerJPMCEndpoints(app, runtime, {
        commerceConfig
    })


    const serveApplePayDomainFile = runtime.serveStaticFile(
        'static/apple-developer-merchantid-domain-association.txt'
    )
    app.get('/.well-known/apple-developer-merchantid-domain-association',
serveApplePayDomainFile)
    app.get(
        '/.well-known/apple-developer-merchantid-domain-association.txt',
        serveApplePayDomainFile
    )
```

14.2 Registered API endpoints

After calling `registerJPMCEndpoints()`, these endpoints are available:

Method	**API endpoint**	**Description**
GET/POST	`/api/jpmorgan/config`	Client-safe configuration
GET/POST	`/api/jpmorgan/googlepay/config`	Google Pay configuration
GET	`/api/jpmorgan/applepay/config`	Apple Pay configuration
GET	`/api/jpmorgan/available-payment-methods`	Enabled payment types
GET	`/api/jpmorgan/fraud-config`	SafeTech Fraud configuration
POST	`/api/jpmorgan/verify`	Card verification

POST	`/api/jpmorgan/authorize`	Payment Authorization
POST	`/api/jpmorgan/applepay/session`	Apple pay merchant validation
POST	`/api/jpmorgan/applepay/authorize`	Apple pay authorization
POST	`/api/jpmorgan/order/:orderNo/confirm`	Confirm order after payment
POST	`/api/jpmorgan/order/:orderNo/fail`	Mark orer as failed

15. CONFIGURE ROUTES

File: `app/routes.jsx`
Purpose: Override default PWA Kit routes with JPMC-integrated page components

Create and override the routes of existing PWA kit pages.
For example – if you have created an override file like index.jsx and payments.jsx in checkout folder

Step 1 – creating the route constant

```
const Checkout = loadable(() => import('./pages/checkout'), {fallback})
```

Step 2 – Adding and exporting the routes.

```
export const routes = [
    // JPMC Override: Checkout with payment provider
    {
        path: '/checkout',
        component: Checkout,
        exact: true
    },
```

16. CHECKOUT PAGE INTEGRATION

16.1 Wrap Checkout with JPMCCheckoutProvider

File: `app/pages/checkout/index.jsx`
Purpose: Initialize JPMC payment context and provide basket mutations to child components

Understanding the Provider

`JPMCCheckoutProvider` is a React context provider that:

1. Initializes the PIE SDK for card encryption

2. Fetches JPMC configuration from Business Manager Site Preferences

3. Manages payment state (loading, errors, card data)

4. Provides functions for card verification and wallet payment authorization

5. Handles basket operations (add/remove payment instruments)

```
    // Get basket mutations
    const {mutateAsync: addPaymentInstrumentToBasket} =
useShopperBasketsMutation('addPaymentInstrumentToBasket')
    const {mutateAsync: updateBillingAddressForBasket} =
useShopperBasketsMutation('updateBillingAddressForBasket')
    const {mutateAsync: removePaymentInstrumentFromBasket} =
useShopperBasketsMutation('removePaymentInstrumentFromBasket')
    const {proxy, organizationId, siteId} = useConfig()


    return (
        <JPMCCheckoutProvider
            locale={locale?.id}
            useBasketHook={() => basketHook}
            useAccessToken={useAccessToken}
             commerceConfig={{proxy, organizationId, siteId}}

commerceConfig={{proxy, organizationId, siteId}}

            config={{
                addPaymentInstrumentToBasket,
                updateBillingAddressForBasket,
                removePaymentInstrumentFromBasket
```

```
            }}
        >
            {/* All checkout components go here */}
            <YourCheckoutContent />
        </JPMCCheckoutProvider>
```

16.2 Placing order integration

Integration Steps –

Step 1 – Import JPMC provider and hooks

```
import {
    JPMCCheckoutProvider,
    useJPMCCheckout,
    useJPMCPlaceOrder,
} from '@jpmorgan/jpmorgan-salesforce-pwa/client'
```

Step 2 – Integrate place order hook for placing order.

```
/**
     * useJPMCPlaceOrder provides:
     * - submitOrder: Function to call when Place Order is clicked
     * - isLoading: True during order creation and payment authorization
     *
     * Configuration:
     * - createOrderFn: Your function that calls SFCC createOrder API
     * - onSuccess: Called with the created order on success
     */
    const {submitOrder, isLoading: isPlacingOrder, error: placeOrderError}
= useJPMCPlaceOrder({
        // createOrderFn: returns the SFCC order
        createOrderFn: async () => {
            const order = await createOrder({body: {basketId:
basket?.basketId}})
            return order
        },
        // onSuccess: navigate to confirmation page
        onSuccess: (order) => {
            navigate(`/checkout/confirmation/${order.orderNo}`)
        },
```

```
    })
```

16.3 Integrating useJPMCCheckout hook for Wallets

Step 1 - Get these required handlers from useJPMCCheckout hook for enabling wallet payments.

```
    const {
        authorizeGooglePayAndPlaceOrder,  // Function for Google Pay
checkout
        isGooglePayProcessing,             // True during Google Pay flow
        authorizeApplePayAndPlaceOrder,   // Function for Apple Pay
checkout
        isApplePayProcessing               // True during Apple Pay flow
    } = useJPMCCheckout()
```

Step 2 – Create Google pay and Apple pay handler functions which will be using our Apple pay and Google pay handler functions that we imported from useJPMCCheckout hook.
These will passed as props to 'payments.jsx' partial file which contains credit card form, wallet buttons etc.

```
const handleGooglePayCheckout = async () => {
        const result = await authorizeGooglePayAndPlaceOrder(
            // createOrderFn - called after Google Pay authorization
succeeds
            () => createOrder({body: {basketId: basket?.basketId}})
        )
        if (result.success) {
            navigate(`/checkout/confirmation/${result.order.orderNo}`)
    }

const handleApplePayCheckout = async () => {
```

```
        const result = await authorizeApplePayAndPlaceOrder(
            () => createOrder({body: {basketId: basket?.basketId}})
        )

        if (result.success) {
            navigate(`/checkout/confirmation/${result.order.orderNo}`)
        }
    }
```

Step 3 – Add Payments.jsx partial file and pass handler as Props.

```
<JPMCCheckoutProvider
    locale={locale?.id}
    useBasketHook={() => basketHook}
    useAccessToken={useAccessToken}
    config={{
        addPaymentInstrumentToBasket,
        updateBillingAddressForBasket,
        removePaymentInstrumentFromBasket
    }}
>
    {/* Other checkout steps... */}
    <ContactInfo />
    <ShippingAddress />
    <ShippingOptions />

    {/* Payment partial - receives wallet handlers as props */}
    <Payment
        onGooglePayCheckout={handleGooglePayCheckout}
        isGooglePayProcessing={isGooglePayProcessing}
        onApplePayCheckout={handleApplePayCheckout}
        isApplePayProcessing={isApplePayProcessing}
    />

    {/* Place Order button */}
    <Button onClick={submitOrder} isLoading={isPlacingOrder}>
        Place Order
    </Button>
</JPMCCheckoutProvider>
```

17. INTEGRATE PAYMENT COMPONENT (CHECKOUT)

File: `app/pages/checkout/partials/payment.jsx`
Purpose: Handle credit card verification and display wallet payment buttons

17.1 Understanding the payment flow

1. User enters card details in form

2. On form submit, `transformPWAKitFormData()` converts form values to JPMC format

3. `verifyAndSavePayment()` calls PIE SDK to encrypt card and verifies with JPMC

4. If verification passes, payment instrument is saved to basket

5. User can then click Place Order (handled in parent component)

17.2 Integration

Step 1 – Import exported functions from library into the payment file.

```
import {
    useJPMCCheckout,          // Hook for payment state and functions
    transformPWAKitFormData, // Utility to transform form values
    GooglePayButton,          // Google Pay button component
    ApplePayButton            // Apple Pay button component
} from '@jpmorgan/jpmorgan-salesforce-pwa/client'
```

Step 2 - Receive wallet handlers as props

The Payment component receives wallet handlers from the parent checkout page:

```
const Payment = ({
    onGooglePayCheckout,
    isGooglePayProcessing,
    onApplePayCheckout,
    isApplePayProcessing
}) => {
    // Component logic here...
}
```

Step 3 - Use useJPMCCheckout hook

Get payment state and functions from the hook (only works inside JPMCCheckoutProvider)

```
const {
    // SDK State
    isReady,               // True when PIE SDK is ready
    isLoading,             // True while PIE SDK is loading
    isVerifying,           // True during card verification
    paymentError,          // Error from verification
    resetPaymentError,     // Clear payment error

    // Payment Functions
    verifyAndSavePayment, // Verify card and save to basket
    refreshKount,          // Refresh Kount fraud session

    // Payment Method Availability
    isCreditCardEnabled,   // True if credit card enabled in BM
    isGooglePayEnabled,    // True if Google Pay available
    isApplePayEnabled,     // True if Apple Pay available
    applePayMerchantId     // Apple Pay merchant ID
} = useJPMCCheckout()
```

Step 4 – **Handle credit card form submission**

```
const handlePaymentFormSubmit = async (formValues, billingAddress) => {
    resetPaymentError()

    // Transform PWA Kit form values to JPMC format
    const cardData = transformPWAKitFormData(formValues, billingAddress)

    // Verify and save payment to basket
    const result = await verifyAndSavePayment(
        addPaymentInstrumentToBasket,
        cardData
    )

    if (result.success) {
        // Payment instrument added - proceed to review step
    }
}
```

Step 5 – Render wallet buttons

```
{isGooglePayEnabled && (
    <GooglePayButton
        autoFetchConfig={true}
        amount={basket?.orderTotal || '0.00'}
        currencyCode={basket?.currency}
        buttonType="checkout"
        disabled={isGooglePayProcessing}
        onClickOverride={onGooglePayCheckout}
    />
)}

{/* Apple Pay Button */}
{isApplePayEnabled && (
    <ApplePayButton
        merchantId={applePayMerchantId}
        amount={basket?.orderTotal || '0.00'}
        currencyCode={basket?.currency}
        buttonType="checkout"
        disabled={isApplePayProcessing}
        onClickOverride={onApplePayCheckout}
    />
)}
```

18. CART EXPRESS CHECKOUT

Enable Apple Pay and Google Pay express checkout buttons on the cart page. This allows customers to complete checkout directly from the cart without going through checkout steps.

Step 1 - Wrap Cart with JPMCCheckoutProvider

File: app/pages/cart/index.jsx

```
// Wrap your cart content
<JPMCCheckoutProvider
    locale={locale?.id}
    useBasketHook={() => basketHook}
    useAccessToken={useAccessToken}
    config={{
        addPaymentInstrumentToBasket,
        updateBillingAddressForBasket,
        removePaymentInstrumentFromBasket,
        // Apple Pay express checkout config
        applePayShippingMethods: initialShippingMethods,
        onApplePayShippingContactSelected: handleShippingContactSelected,
        onApplePayShippingMethodSelected: handleShippingMethodSelected,
        applePayPrePopulateContactFromBasket: false
    }}
>
    <BaseCart {...props} />
</JPMCCheckoutProvider>
```

Step 2 - Import hooks and buttons

File - app/pages/cart/partials/cart-cta.jsx

```
import {
    useJPMCCheckout,
    GooglePayButton,
    ApplePayButton
} from '@jpmorgan/jpmorgan-salesforce-pwa/client'
import {useShopperOrdersMutation} from '@salesforce/commerce-sdk-react'
```

Step 3 - Get payment state and create handlers

```
const {
    isGooglePayEnabled,
    isApplePayEnabled,
    applePayMerchantId,
    authorizeGooglePayAndPlaceOrder,
    authorizeApplePayAndPlaceOrder,
    isGooglePayProcessing,
    isApplePayProcessing
} = useJPMCCheckout()

const {mutateAsync: createOrder} = useShopperOrdersMutation('createOrder')

const handleGooglePayCheckout = async () => {
    const result = await authorizeGooglePayAndPlaceOrder(
        () => createOrder({body: {basketId: basket?.basketId}})
    )
    if (result.success) {
        navigate(`/checkout/confirmation/${result.order.orderNo}`)
    }
}

const handleApplePayCheckout = async () => {
    const result = await authorizeApplePayAndPlaceOrder(
        () => createOrder({body: {basketId: basket?.basketId}})
    )
    if (result.success) {
        navigate(`/checkout/confirmation/${result.order.orderNo}`)
    }
}
```

Step 4 - Render buttons conditionally

```
{isGooglePayEnabled && (
    <GooglePayButton
        autoFetchConfig={true}
        amount={basket?.orderTotal || '0.00'}
        currencyCode={basket?.currency}
```

```
        buttonType="checkout"
        disabled={isGooglePayProcessing}
        onClickOverride={handleGooglePayCheckout}
        style={{width: '100%', minHeight: '48px'}}
    />
)}
{isApplePayEnabled && (
    <ApplePayButton
        merchantId={applePayMerchantId}
        amount={basket?.orderTotal || '0.00'}
        currencyCode={basket?.currency}
        buttonType="checkout"
        buttonStyle="black"
        disabled={isApplePayProcessing}
        onClickOverride={handleApplePayCheckout}
        fullWidth
    />
)}
```

19. PDP EXPRESS CHECKOUT

Enable Apple Pay and Google Pay express checkout buttons on the Product Detail Page. This allows customers to purchase directly from the PDP without adding to cart first.

19.1 Express checkout flow for PDP

- Customer clicks Apple Pay or Google Pay button on PDP
- A temporary basket is created with the selected product
- Payment sheet opens with shipping selection
- Totals update dynamically as customers select shipping
- Payment is authorized by JPMC
- Order is created and customer redirects to confirm
- On cancel/failure, temporary basket is automatically deleted

19.2 Express checkout integration (Apple pay and Google Pay)

Step 1 – Create a PDP override page

Step 2 – Import these required hooks and functions

```
import {
    useJPMCCheckout,
    ApplePayButton,
    GooglePayButton,
    JPMC_ERROR_ROUTE
} from '@jpmorgan/jpmorgan-salesforce-pwa/client'
```

Step 2 - Get state from hook (both wallets)

```
const {
    // Apple Pay
    isApplePayEnabled,
    isApplePayProcessing,
```

```
    applePayMerchantId,
    applePayPaymentMethodId,
    authorizeApplePayAndPlaceOrder,
    // Google Pay
    isGooglePayEnabled,
    isGooglePayProcessing,
    googlePayConfig,
    authorizeGooglePayAndPlaceOrder
} = useJPMCCheckout()
```

Step 3 – Create handler functions and Call authorization functions in your handlers

```
// Apple Pay handler
const result = await
authorizeApplePayAndPlaceOrder(createOrderFromTempBasket, {
    amount: unitPrice * quantity,
    currencyCode: currency,
    shippingMethods: [{identifier: 'loading', label: 'Calculating...',
amount: '0.00'}]
})

// Google Pay handler (inside onPaymentAuthorized callback)
const result = await authorizeGooglePayAndPlaceOrder(
    () => createOrderFromTempBasket(paymentData),
    paymentData,
    {amount: orderTotal, currencyCode: currency}
)

// Both - on success
if (result.success) {
    navigate(`/checkout/confirmation/${result.order.orderNo}`)
}
```

Step 4 – Render Google and Apple pay buttons

```
{isApplePayEnabled && (
    <ApplePayButton
        merchantId={applePayMerchantId}
        buttonType="buy"
        buttonStyle="black"
        disabled={isApplePayProcessing}
        onClickOverride={handleApplePayPress}
        fullWidth
    />
)}

{isGooglePayEnabled && (
    <GooglePayButton
        context="pdp"
        gatewayMerchantId={googlePayConfig?.gatewayMerchantId}
        merchantName={googlePayConfig?.merchantName}
        amount={String(price * quantity)}
        currencyCode={currency}
        shippingAddressRequired={true}
        billingAddressRequired={true}
        emailRequired={true}
        onPaymentDataChanged={handlePaymentDataChanged}
        onPaymentAuthorized={handlePaymentAuthorized}
        onCancel={handleCancel}
        onError={handleError}
        buttonType="buy"
        buttonColor="black"
        disabled={isGooglePayProcessing}
    />
)}
```

20. MULTI MERCHANT CONFIGURATION

20.1 What is Multi-Merchant Functionality?

By default, the cartridge uses a single JPMC merchant account for all transactions. The multi-Merchant feature allows you to override this and assign a unique set of JPMC gateway credentials to each locale (e.g., your Canadian English site, your US site, etc.).

This means each locale can have its own independent:

- *Merchant ID (MID)*
- *Fraud and AVS rules*
- *Currency processing rules*
- *Google Pay settings*

Example:
If a shopper uses this locale... | ...their payment is processed by this account:
en_CA (English Canada) | Canada (CAD) account
en_US (English US) | US (USD) account

20.2 When Do You Need It? A Checklist

You may use this feature if any of the following apply to your business:

- *You operate in multiple countries and have separate merchant accounts for each.*
- *You process payments in multiple currencies, and your acquirer requires a different MID per currency.*
- *You need to apply different fraud rules (e.g., AVS requirements) for specific regions.*
- *You need to use distinct Google Pay merchant identifiers for different countries.*

- *If you operate in a single country with a single currency, you do not need this feature.*

20.3 Setup Process

Part 1: The Setup Process (5 Steps)

Follow these five steps to configure a locale with its own merchant account.

Step 1: Gather Your Credentials

Before you begin, ensure you have the following information from JPMC for each locale you plan to configure:

- *Merchant ID (MID)*
- *Client ID*
- *PIE Key*

Step 2: Enable the Feature

First, you must activate the functionality for your site.

- *Navigate to:* ***Merchant Tools** → **Site Preferences** → **Custom Preferences.***
- *Select the JPMC Multi Merchant group.*
- *Find Enable Multi-Merchant and set it to Yes.*
- *Click Save.*

This setting can be turned off at any time to revert to using a single, site-wide merchant account without losing your saved configurations.

Step 3: Create a New Locale Configuration

To See **JPMC Tools > Multi-Merchant Configuration in Merchant Tools**

- ***Administration > Organization > Roles & Permissions > SELECT A ROLE (Eg: Administrator) > Business Manager Modules***
- *Select Site (context)*
- *Click Multi-Merchant Configuration*
- *Click Update*

Business Manager Module	Module Description		
JPMC Tools			☑
Certificate Thumbprint Generator	Generate SHA-1 thumbprint from BM Certificate Manager		☑
Multi-Merchant Configuration	Create and Manage per-locale JPMC merchant configurations		☑

Next, you will create the configuration for a specific locale.

- *Navigate to: JPMC Tools → Multi-Merchant Configuration.*
- *Click the + Create / Edit Configuration button.*
- *From the Select Locale dropdown, choose the locale you want to configure (e.g., en_US,en_CA)*

Merchant Configurations

About Merchant Configuration

This tool allows you to manage locale-specific payment gateway configurations for your JPMC integration. Each locale (e.g., en_US, en_CA) can have its own merchant settings, including client credentials, tokenization preferences, Google Pay and Apple Pay configurations, and fraud detection settings.

Default locale settings are managed separately through Site Preferences.

+ Create / Edit Configuration | Refresh All Cache | **2** Configuration(s) found

Locale	Merchant ID	Actions
en_CA	[redacted]	Edit Configuration | Delete Configuration
en_US	[redacted]	Edit Configuration | Delete Configuration

Step 4: Fill in the Configuration Details

Now, enter the specific credentials for the selected locale.

- *Key Principle: You only need to fill in fields that are different from your site's default settings. Any field you* *leave blank will automatically use the value from your main Site Preferences.*

- *Required Field: Merchant ID is the only mandatory field.*
- *Security Note: Sensitive fields will be masked (******) after saving for security. If you don't change them, the existing saved value is preserved.*

Step 5: Save and Verify Your Setup

- *Click Save Configuration. Your new settings are now active.*
- *To verify, place a test order on your storefront using the newly configured locale. Confirm the payment is processed successfully and check the order details in Business Manager to ensure the correct Merchant ID was used.*

20.4 Management

Part 2: Management

The **JPMC Tools → Multi-Merchant Configuration** page is your central hub for managing all setups.

Action	How to Do It
Edit a Configuration	Click Edit Configuration next to the locale you wish to update. Changes are live immediately after you save.
Temporarily Disable	Edit the configuration and set the Enabled toggle to No. The locale will temporarily use the site's default account.
Delete a Configuration	Click Delete Configuration. The locale will permanently revert to using the site's default account.

Force a Cache Update	Click Refresh All Cache on the main list page to ensure all storefront servers have the latest settings.

20.5 Key Concepts

Part 3: Key Concepts Explained

How "Fallback" Simplifies Your Work

You do not need to fill in every single field for every locale. The system uses a smart "fallback" logic:

- *It first looks for a setting in your Locale-specific Configuration.*
- *If the field is blank there, it falls back to the value in your main Site Preferences.*
- *If it's blank there too, it uses the built-in system default.*

This saves time and reduces duplication. You only need to define what's different.

<u>The Importance of Order-Level Tracking</u>

When an order is placed, the system saves the Merchant ID that was used directly on the order itself. This is critical because it means changes to your configurations do not affect past orders. If you need to capture or refund an old order, the system will always use the correct credentials that were active when the order was first placed.

20.6 Troubleshooting Multi-Merchant Issues

Part 4: Troubleshooting Common Issues

If you see this problem...	...try this solution:
A locale is using the wrong account.	Go to its configuration and ensure the Enabled toggle is set to Yes. Then, click the Refresh All Cache button on the main configuration page.
The "Create / Edit" page shows "Disabled".	The main feature is turned off. Follow the instructions in Part 1, Step 2 to enable it in Site Preferences.
I deleted a configuration by mistake.	Don't worry, this does not affect past orders. New orders on that locale will now use the site's default credentials. You can recreate the configuration at any time.
The "Select Locale" dropdown is empty.	Your site has no locales configured other than default. You must first add locales to your site in Merchant Tools → Site → Site Settings → Locales.

20.7 Passing locale to Checkout provider

Pass locale to `JPMCCheckoutProvider`

```jsx
import useMultiSite from '@salesforce/retail-react-app/app/hooks/use-multi-site'

const CheckoutContainer = () => {
```

````
    const { locale } = useMultiSite()

    return (
        <JPMCCheckoutProvider
            locale={locale?.id}  // e.g., 'en_CA', 'fr_FR'
            useBasketHook={() => basketHook}
            useAccessToken={useAccessToken}
        >
            <Checkout />
        </JPMCCheckoutProvider>
    )
}
```
````
````

21. FRAUD CHECK (SAFETECH)

21.1 Overview

The library integrates with Kount for device fingerprinting. When enabled:

1. Kount SDK loads automatically when `JPMCCheckoutProvider` mounts

2. Device data is collected under a session ID.

3. Session ID is sent in the Fraud Check request.

4. JPMC evaluates fraud risk and returns action (Accept/Review/Decline)

21.2 Enable Fraud Check

- ***Merchant Tools > Site Preferences > Custom Preferences***
- *Open JPMC Payment Configuration*
- *Set Enable JPMC Fraud Check = `true`*
- *Click Save*

17.3 Fraud Rule Action

ACTION	BEHAVIOR
A	Accept. Proceed to authorization
E, R	Proceed; flag for manual review
D	Decline before authorization

22. ENABLE 3D SECURE (OPTIONAL)

22.1 Enable 3DS

- ***Merchant Tools > Site Preferences > Custom Preferences***
- *Open JPMC 3D Secure Configuration*
- *Set Enable JPMC 3D Secure = `true`*

Click Save

22.2 Multi-Merchant Configuration for 3DS

- *If your storefront operates in multi-merchant mode, you should enable 3DS flag independently for each merchant locale*
- *Default Value: False*
-

****See the Multi Merchant section for reference.**

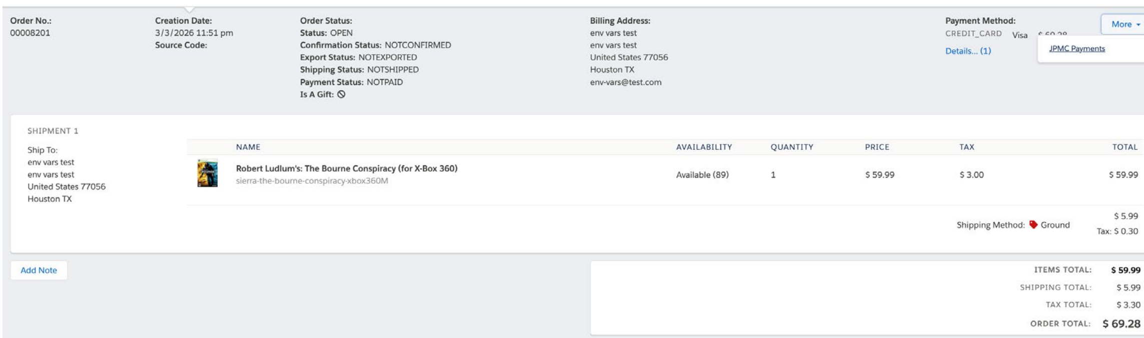
23. ORDER MANAGEMENT IN BUSINESS MANAGER (CSC)

23.1 Access JPMC Payments Tab

1. **Merchant Tools** > **Ordering** > **Orders**

2. Open an order

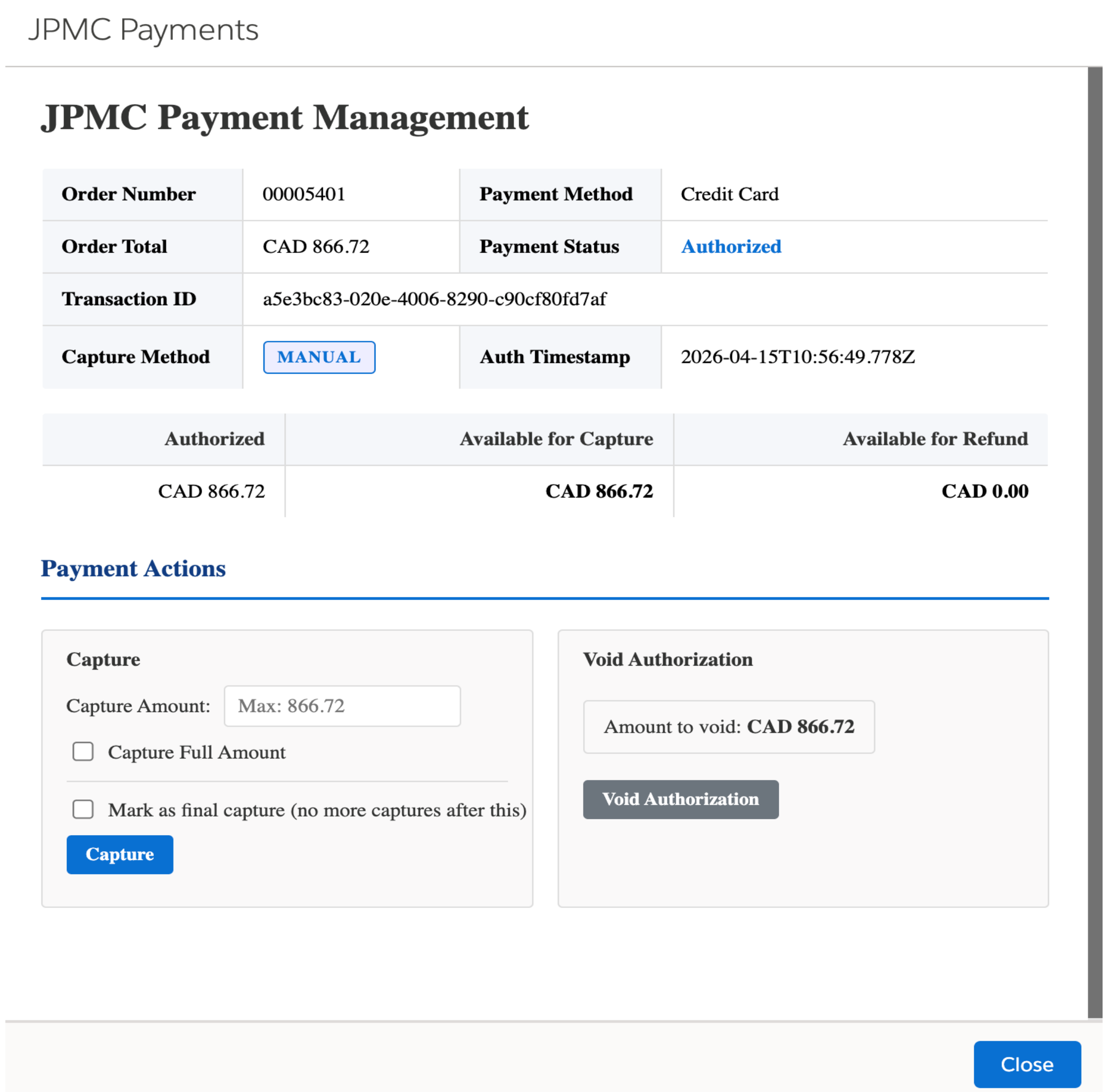
3. Select **JPMC Payments** tab



23.2 Available Actions

ACTION	DESCRIPTION	WHEN AVAILABLE
Capture	Capture authorized funds (full amount)	After authorization (MANUAL/DELAYED)
Refund	Refund captured payment (full/partial)	After capture
Void	Cancel authorization before capture	After authorization, before capture

Note: If capture method is `NOW`, capture occurs at authorization and **Capture** is not shown.


JPMC Payments
JPMC Payment Management
Order Number
00005401
Payment Method
Credit Card
Order Total
CAD 866.72
Payment Status
Authorized
Transaction ID
a5e3bc83-020e-4006-8290-c90cf80fd7af
Capture Method
MANUAL
Auth Timestamp
2026-04-15T10:56:49.778Z
Authorized
Available for Capture
Available for Refund
CAD 866.72
CAD 866.72
CAD 0.00
Payment Actions
Capture
Capture Amount:
Max: 866.72
Capture Full Amount
Mark as final capture (no more captures after this)
Capture
Void Authorization
Amount to void: CAD 866.72
Void Authorization
Close

23.2.1 Capture a Payment

*When to use: If your capture method is `MANUAL`, the payment was only *authorized* (funds reserved) at checkout. You now need to *capture* (charge) the customer — typically when you ship the product.*

- *Open the order in Business Manager.*
- *Navigate to the **JPMC Payments** section.*
- *You'll see the **authorized amount** and the **available for capture** amount.*
- *Enter the amount to capture (the full amount, or a partial amount for partial shipments).*
- *Click **Capture**.*
- *The payment status updates. You can capture multiple times until the full authorized amount is used.*

Payment Status	*What It Means*
***A** (Authorized)*	*Payment approved, funds reserved, nothing charged yet.*
***AC** (Auth & Captured)*	*Payment was charged immediately at checkout (using `NOW` mode).*
***C** (Captured)*	*Full authorized amount has been captured.*
***PC** (Partial Captured)*	*Some of the authorized amount has been captured; more remains.*

23.2.2 Issue a Refund

- *When to use: A customer wants their money back — either for a return, a complaint, or an error.*
- *Open the order.*
- *In the JPMC Payments section, you'll see the captured amount and the available for refund amount.*
- *Enter the amount to refund (full or partial).*
- *Click **Refund**.*
- *The refund is processed through J.P. Morgan, and the money is returned to the customer's card.*

Payment Status	What It Means
***RF** (Refunded)*	The full captured amount has been refunded.
***PRF** (Partial Refunded)*	Some of the captured amount has been refunded; more remains.

23.2.3 Void a Transaction

When to use: You want to cancel an authorized payment before any funds have been captured (e.g., the customer calls to cancel before you ship).

- *Open the order.*
- *In the JPMC Payments section, click Void*
- *The authorization is cancelled. No funds are captured.*

***Important: ** You can only void a payment that has **not yet been captured**. Once funds are captured, you must issue a refund instead.*

23.3 Transaction History

Every action (capture, refund, void) is logged in the order's JPMC transaction history. You can view:

- *Transaction ID*
- *Amount*
- *Timestamp*
- *Status*
- *Who performed the action*

This provides a complete audit trail for your records.

24. TESTING AND VALIDATION

24.1 Test Card payments

Use J.P. Morgan sandbox test cards per J.P. Morgan developer documentation.

https://developer.payments.jpmorgan.com/docs/commerce/online-payments/testing

24.2 Test scenarios

Scenario	Expected Result
Successful card payment	Order placed, confirmation page shown
Declined card	Error shown on checkout, no order created
Auth fail after order	Order marked failed, order-failed page shown
Google Pay success	Order placed via Google Pay
Apple Pay success	Order placed via Apple Pay
User cancels wallet	Stays on checkout, no error
PIE SDK load failure	Error alert shown on payment step

24.3 Debug mode

Enable debug logging –

```bash
# In .env
JPMC_DEBUG=true
```

This logs:

- Configuration resolution
- API requests/responses

- PIE SDK loading
- Payment flow steps

25. GO LIVE CHECKLIST

25.1 Credentials

- *Obtain production credentials from JPMC*
- *Upload production certificate and private key to MRT*
- *Update environment variables with production values*

25.2 Business Manager

- *Update Site Preferences with production values*
- *Set `JPMCEnvironment` to `production`*
- *Set `JPMCGooglePayEnvironment` to `PRODUCTION`*
- *Set `JPMCKountEnvironment` to `PRODUCTION` (if using fraud)*
- *Verify payment methods are enabled*

25.3 Testing

- Complete end-to-end card payment
- Complete end-to-end Google Pay payment
- Complete end-to-end Apple Pay payment
- Verify order confirmation displays correctly
- Verify order-failed page works
- Verify orders appear in Business Manager

25.4 Security

- Verify PIE encryption is working (no plain card numbers in logs)
- Verify HTTPS is enforced
- Verify CSP headers are present

Sensitive credentials only in MRT environment variables

26. TROUBLESHOOTING

26.1 PIE SDK Not loading

Symptom	Possible Cause	Resolution
"Initializing secure payment..." stuck	CSP blocking PIE scripts	Verify `jpmorganCSPMiddleware()` is applied
"Failed to load PIE encryption SDK"	Invalid PIE URLs	Check `JPMCPIEGetKeyUrl` and `JPMCPIEEncryptionUrl` site preferences in Business manager
PIE ready but encryption fails	Wrong merchant ID in getkey URL	erify PIE merchant ID matches

26.2 Authorization failures

Symptom	Possible Cause	Resolution
Invalid token	Certificate/key mismatch	Re-upload matching cert and key
Unauthorized	Wrong client ID/resource ID	Verify Site Preferences
Network timeout	API host incorrect	Check `JPMCApiHost` value

26.3 Google pay not showing

Symptom	Possible Cause	Resolution
Button not rendered	Google Pay disabled in BM	Enable `JPMCGooglePayEnabled`
Google Pay unavailable	No cards in wallet	Add cards to Google account

Script load error	CSP blocking Google domains	Verify CSP includes Google domains

26.4 Apple pay not showing

Symptom	**Possible Cause**	**Resolution**
Button not rendered	Not Safari browser	Apple pay works only in safari browser
Apple Pay unavailable	No cards in wallet	Add test cards to Apple wallet
Merchant validation fails	Wrong merchant ID	Verify `JPMCApplePayMerchantId`

26.5 Viewing logs

MRT logs can be viewed in terminal using command –

```
npx @salesforce/b2c-cli@latest mrt tail-logs --project test-project-1 --
environment dev
```

27. SUPPORT & ESCALATION

27.1 Resources

- **JPMC Developer Portal:** https://developer.payments.jpmorgan.com

- **PWA Kit Documentation:** https://developer.salesforce.com/docs/commerce/pwa-kit-managed-runtime/overview

27.2 Contact

- **JPMC API Issues:** Contact your J.P. Morgan technical account representative

- **Library Issues:** Contact your Salesforce Commerce Cloud system integrator

- **PWA Kit Issues:** Salesforce Commerce Cloud support

APPENDICES

Appendix A – Complete ssr.js example

````
```javascript
import dotenv from 'dotenv'
import path from 'path'

// Load .env for LOCAL DEVELOPMENT only
// In MRT, environment variables are set in the MRT dashboard
const envPath = path.resolve(process.cwd(), '.env')
dotenv.config({ path: envPath })

import { getRuntime } from '@salesforce/pwa-kit-
runtime/ssr/server/express'
import { getConfig } from '@salesforce/pwa-kit-runtime/utils/ssr-config'
import bodyParser from 'body-parser'
import { registerJPMCEndpoints, jpmorganCSPMiddleware } from
'@jpmorgan/jpmorgan-salesforce-pwa/ssr'
npx @salesforce/b2c-cli@latest mrt tail-logs --project test-project-1 --
environment dev
const options = {
    buildDir: path.resolve(process.cwd(), 'build'),
    defaultCacheTimeSeconds: 600,
    mobify: getConfig(),
    port: 3000,
    protocol: 'http'
}

const runtime = getRuntime()

// Commerce config for Order API
// Note: Site Preferences service reads directly from process.env,
// so all COMMERCE_API_* environment variables must be set
const siteConfig = getConfig()?.app?.commerceAPI?.parameters || {}

const { handler } = runtime.createHandler(options, (app) => {
    app.use(bodyParser.json())
    app.use(jpmorganCSPMiddleware())

    registerJPMCEndpoints(app, runtime, {
        debug: process.env.JPMC_DEBUG === 'true',
        commerceConfig: {
````
````

```
            orgId: siteConfig.organizationId ||
process.env.COMMERCE_API_ORG_ID,
            shortCode: siteConfig.shortCode ||
process.env.COMMERCE_API_SHORT_CODE,
            siteId: siteConfig.siteId || process.env.COMMERCE_API_SITE_ID,
            clientId: process.env.COMMERCE_API_CLIENT_ID_PRIVATE,
            clientSecret: process.env.COMMERCE_API_CLIENT_SECRET
        }
    })

    app.get('/robots.txt', runtime.serveStaticFile('static/robots.txt'))
    app.get('*', runtime.render)
})

export const get = handler
```

Appendix B – Complete checkoutContainer example –

```jsx
import React, { useState } from 'react'
import { useCurrentBasket } from '@salesforce/retail-react-
app/app/hooks/use-current-basket'
import {
    useAccessToken,
    useShopperBasketsMutation,
    useShopperOrdersMutation
} from '@salesforce/commerce-sdk-react'
import useMultiSite from '@salesforce/retail-react-app/app/hooks/use-
multi-site'
import { CheckoutProvider } from '@salesforce/retail-react-
app/app/pages/checkout/util/checkout-context'
import { JPMCCheckoutProvider } from '@jpmorgan/jpmorgan-salesforce-
pwa/client'
import CheckoutSkeleton from '@salesforce/retail-react-
app/app/pages/checkout/partials/checkout-skeleton'

const CheckoutContainer = () => {
    const basketHook = useCurrentBasket()
    const { data: basket } = basketHook
    const { locale } = useMultiSite()
```

```
    const { mutateAsync: addPaymentInstrumentToBasket } =
useShopperBasketsMutation(
        'addPaymentInstrumentToBasket'
    )
    const { mutateAsync: updateBillingAddressForBasket } =
useShopperBasketsMutation(
        'updateBillingAddressForBasket'
    )
    const { mutateAsync: removePaymentInstrumentFromBasket } =
useShopperBasketsMutation(
        'removePaymentInstrumentFromBasket'
    )
     const {proxy, organizationId, siteId} = useConfig()



    return (
        <CheckoutProvider>
            <JPMCCheckoutProvider
                locale={locale?.id}
                useBasketHook={() => basketHook}
                useAccessToken={useAccessToken}
               commerceConfig={{proxy, organizationId, siteId}}


                config={{
                    addPaymentInstrumentToBasket:
addPaymentInstrumentToBasket,
                    updateBillingAddressForBasket:
updateBillingAddressForBasket,
                    removePaymentInstrumentFromBasket:
removePaymentInstrumentFromBasket,
                    applePayShippingAddressRequired: false
                }}
            >
                {!basket?.basketId ? (
                    <CheckoutSkeleton />
                ) : (
                    <Checkout />
                )}
            </JPMCCheckoutProvider>
        </CheckoutProvider>
    )
```

```
}

export default CheckoutContainer
```

Document Version: 1.0 | **Package Version:** 1.0.0

Appendix C – Steps to Create SLAS client

- Follow the documentation and create SLAS client https://help.salesforce.com/s/articleView?id=cc.b2c_shopping_agent_create_private_slas_client.htm&type=5

- For the App Type, choose SPA or Mobile (Public).
- Choose tenant (Sandbox), add site RefArch.
- Choose standard scopes.

Appendix D – Base 64 Encoding using OpenSSL

```
# On Windows PowerShell
# Convert DER to PEM first
openssl x509 -inform DER -in certs\your-certificate.cer -out certs\your-certificate.pem

# Then convert to base64
[Convert]::ToBase64String([IO.File]::ReadAllBytes("certs\your-certificate.pem"))
```